

CMS + CountSketch

Guion de exposición y guía breve del código

Tema

Detección de ataques de red con sketches de frecuencia sobre una ventana deslizante de 60 segundos. La exposición se organiza en 9 diapositivas y dura cerca de 8 minutos, más el tiempo para preguntas.

Integrante	Diapositivas	Tiempo	Tema
1	1-3	3:00	Contexto y diseño
2	4-6	2:30	Precisión y detección
3	7-9	2:30	Cambio entre ventanas y conclusiones

Cifras de referencia

La traza MAWI contiene 123 383 971 paquetes IPv4. Los ataques comienzan en $t=300$ s y duran 30 s. Se usa semilla 42 para generar los ataques. Las semillas de hash son 42, 7 y 1234. En las diapositivas de resultados se muestra la semilla de hash 42.

Al exponer, no confundir la semilla de generación de ataques con la semilla de las funciones hash.

1. Qué hace el experimento

El programa estima cuántos paquetes de una IP aparecen en los últimos 60 segundos, con una evaluación cada 10 segundos. Una IP se considera heavy hitter cuando concentra más del 1 % de los paquetes de la ventana. La estimación se compara con un conteo exacto.

Se usa una traza de MAWI samplepoint-F del 3 de diciembre de 2018 a las 14:00. Sobre ella se inyectan dos ataques sintéticos: un DDoS dirigido a 163.210.30.13 y un scan originado en 198.18.0.7. Para el DDoS se cuenta la IP de destino; para el scan, la IP de origen.

Flujo de datos

- La captura 201812031400.pcap.gz se convierte a registros binarios de 24 bytes mediante pcap2bin.cpp.
- inject_attack.py mezcla los ataques sintéticos con la traza y escribe los archivos de ground truth. La semilla de inyección es 42.
- exact_hh.cpp calcula la referencia exacta por ventana. sliding_sketch.cpp estima con CMS y CS para cada ancho y semilla de hash.
- analisis.py cruza las salidas, calcula errores y decisiones de detección, y genera tablas y figuras en codigo_entregado/resultados/.
- run_experimentos.sh coordina la preparación de trazas y las corridas.

La captura de MAWI se descarga desde la URL indicada en la sección de fuentes. No se sube al repositorio.

2. Cómo funciona la ventana

La ventana de 60 s se divide en seis subventanas de 10 s. Cada una tiene un sketch; un séptimo sketch mantiene la suma de las seis subventanas. Las ranuras se reutilizan en anillo.

Cuando comienza una subventana, se resta del agregado la ranura que contiene los paquetes que ya salen de la ventana. Después se limpia esa ranura y se reutiliza para recibir los paquetes nuevos. Gracias a la linealidad de los sketches, actualizar la ventana cuesta $2 \cdot d \cdot w$ contadores por rotación, sin recorrer de nuevo los paquetes.

CMS actualiza contadores positivos y estima con el mínimo de sus filas. Las colisiones sólo pueden aumentar la estimación. CountSketch actualiza con signos +1 o -1 y estima con la mediana; las colisiones pueden producir errores en ambos sentidos.

Para Δf , CountSketch usa el sketch firmado de la diferencia entre ventanas. CMS no conserva su garantía al trabajar con contadores negativos; en el experimento se usa la mediana como estimador experimental.

3. Guion por diapositiva

Integrante 1 · Contexto y diseño · aproximadamente 3 minutos

Diapositiva 1 · CMS + CountSketch · 0:30

Buenas tardes. En este trabajo estudiamos cómo estimar cuántos paquetes tuvo una IP durante los últimos 60 segundos, sin guardar todos los paquetes ni volver a recorrerlos. Con esa frecuencia identificamos posibles ataques. Comparamos dos sketches, Count-Min y CountSketch, con un conteo exacto.

Diapositiva 2 · Trazo MAWI y ataques sintéticos · 0:45

Usamos una captura real de MAWI, de 15 minutos y unos 123 millones de paquetes IPv4. Le agregamos dos ataques de 30 segundos, desde el segundo 300. En el DDoS, 4 000 fuentes envían tráfico a una víctima y contamos por destino. En el scan, una IP contacta 60 000 destinos y contamos por origen. La ventana es de 60 segundos y avanza cada 10. El umbral de detección es el 1 % de los paquetes de la ventana. Probamos cuatro anchos, desde 256 hasta 16 384. La semilla usada para generar los ataques es 42.

Diapositiva 3 · Ventana deslizante por linealidad · 1:45

Esta es la parte central del diseño. Dividimos la ventana en seis subventanas de 10 segundos y guardamos sus sketches en un anillo. También mantenemos A, la suma de las seis, que representa la ventana completa. Cada paquete se agrega a su ranura y a A. Como el sketch es lineal, al avanzar la ventana restamos la subventana antigua, limpiamos su ranura y la reutilizamos. La rotación toca 2 por d por w contadores, sin importar cuántos paquetes haya en la ventana. La misma resta permite calcular el cambio entre ventanas. Validamos la alineación comparando el total de paquetes con el programa exacto: coincidieron las 84 ventanas.

Traspaso: “Ahora [Integrante 2] muestra qué tan precisos son los sketches y cómo se comportan al detectar los ataques”.

Integrante 2 · Resultados de frecuencia · aproximadamente 2 minutos 30 segundos

Diapositiva 4 · El error baja al aumentar el ancho · 0:45

Primero medimos el error sin ataque para las claves de frecuencia alta. Al aumentar el ancho, el error baja en CMS y CS. En estos grupos, CS tuvo menor error con la misma memoria. CMS sobreestima porque las colisiones se suman; CS usa signos aleatorios, por lo que algunos errores se cancelan y la estimación puede quedar por encima o por debajo. La memoria va desde 35 KiB con ancho 256 hasta 2 240 KiB con ancho 16 384.

Diapositiva 5 · DDoS: ambos sketches igualan la detección exacta · 0:45

En el DDoS, el conteo exacto detecta a la víctima en 310 segundos, diez segundos después del inicio. CMS y CS la detectan en esa misma evaluación para todos los anchos, sin falsos positivos. Incluso con ancho 256 el error relativo medio es 7,56 % en CMS y 3,89 % en CS. Al terminar el ataque, el conteo va bajando a medida que sus paquetes salen de la ventana.

Diapositiva 6 · Scan: con w=256 aparecen falsos positivos · 1:00

El scan presenta un caso más difícil. La referencia exacta lo detecta a los 320 segundos, con 20 segundos de latencia. Para la semilla de hash 42 y ancho 256, CMS y CS cruzan el umbral a los 310 segundos, antes que la referencia, y vuelven a producir un falso positivo a los 380. Desde ancho 1 024, las decisiones coinciden con la referencia: 20 segundos de latencia y cero falsos positivos. Con ancho 1 024, el error relativo medio es 0,543 % para CMS y 0,127 % para CS.

Traspaso: “Ahora [Integrante 3] muestra qué información entrega el cambio entre ventanas”.

Integrante 3 · Cambio entre ventanas y cierre · aproximadamente 2 minutos 30 segundos

Diapositiva 7 · DDoS: Δf señala entrada y salida · 0:40

La frecuencia de una ventana reacciona gradualmente porque acumula 60 segundos. El cambio entre ventanas es más directo: aumenta 100 008 paquetes a los 310 segundos, cuando entra el ataque, y disminuye 100 007 a los 370, cuando sale de la ventana. CountSketch estima la diferencia con su mecanismo de signos. Para CMS usamos la mediana como variante experimental; al haber contadores negativos, no se mantiene la garantía habitual del CMS. Con ancho 16 384, los extremos estimados quedan a dos paquetes o menos de los valores exactos.

Diapositiva 8 · Scan: el estimador depende del extremo · 0:40

En el scan, el incremento exacto es de 80 000 paquetes y el decremento también es de 80 000 en valor absoluto. Con ancho 256, CMS-mediana estima mejor el incremento, con 78 paquetes de error frente a 133 de CS. Para el decremento, CS tiene 155 de error y CMS-mediana 417. Por eso no hay un estimador que sea mejor en ambos extremos con ese ancho. Al aumentar el ancho, los errores disminuyen.

Diapositiva 9 · Conclusiones · 1:10

Nos quedamos con cuatro conclusiones. Primero, la linealidad permite mantener la ventana deslizante con un costo de $2dw$ contadores por rotación. Segundo, aumentar el ancho reduce el error; en general, CS fue más preciso en los casos evaluados, mientras CMS conserva un sesgo hacia arriba. Tercero, el DDoS se detecta igual que con el conteo exacto; el scan con ancho 256 presenta falsos positivos, y desde 1 024 las decisiones coinciden con la referencia. Cuarto, Δf muestra cuándo el ataque entra y cuándo sale de la ventana. Gracias. Quedamos atentos a sus preguntas.

4. Preguntas probables

¿Por qué no reiniciar un único sketch cada 60 segundos?

Eso produciría ventanas fijas. Inmediatamente después de un reinicio se perdería la historia reciente. El anillo mantiene siempre los últimos 60 segundos.

¿Cuánta memoria adicional requiere la ventana?

Se guardan seis sketches de subventana y un agregado. La memoria reportada va de 35 KiB a 2 240 KiB según el ancho, con $d=5$ y contadores de cuatro bytes.

¿Cómo verificaron que las ventanas están alineadas con `exact_hh`?

El programa compara el número de paquetes de cada ventana con `exact_hh` y termina con error si encuentra una diferencia. En las corridas, coincidieron las 84 ventanas.

¿Por qué CMS puede tener falsos positivos?

Las colisiones se suman, de modo que CMS puede sobreestimar. Si la estimación cruza el umbral antes que el conteo real, aparece un falso positivo.

¿Por qué DDoS tiene 10 s de latencia y scan 20 s?

Se evalúa cada 10 segundos. El DDoS supera el umbral en la primera evaluación posterior al inicio; el scan lo supera en la segunda según el conteo exacto.

¿Qué pasa con ataques de menor intensidad?

Con 3 000 paquetes por segundo, ambos ataques se detectaron con 30 segundos de latencia, igual que la referencia exacta y sin falsos positivos en las configuraciones informadas. El error crece, sobre todo con ancho 256.

¿Los resultados dependen de la semilla de hash?

Se repitieron las corridas con semillas 7, 42 y 1234. La tendencia general se mantiene; CountSketch presenta más variación entre semillas que CMS.

¿Por qué CMS-mediana no conserva la garantía del CMS en Δf ?

La garantía usual requiere contadores no negativos y estimación mediante el mínimo. La diferencia entre ventanas produce valores negativos, así que la mediana usada en el experimento es sólo una variante.

Fuentes y entrega

Traza MAWI: <https://mawi.wide.ad.jp/mawi/samplepoint-F/2018/201812031400.pcap.gz>

Repositorio del proyecto: <https://github.com/BisenteWiederhold/Tarea1TopMGVD>

La traza no se incluye en el repositorio. Semilla para generar los ataques: 42. Semillas de hash probadas: 7, 42 y 1234.